```
import numpy as np
import random
def evaluate_cost(replenish_qty, reorder_pt, zone_alloc, demand, holding_rate, setup_cost, travel_distances, zone_capacities, ite
    """
    Calculates the total operational cost of a specific particle configuration.
    Goal: Minimize this value.
    """
    # 1. Holding Cost = (Average Cycle Stock + Safety Stock) * Unit Holding Rate
    holding_cost = ((replenish_qty / 2.0) + reorder_pt) * holding_rate

    # 2. Ordering Cost = (Annual Demand / Order Quantity) * Setup Fee
    # Prevent division by zero if quantity drops to zero
    if replenish_qty > 0:
        ordering_cost = (demand / replenish_qty) * setup_cost
    else:
        ordering_cost = 999999.0

    # 3. Handling Cost = Demand * Distance from allocated zone to shipping dock
    # zone_alloc is 1-indexed for the user, convert to 0-indexed for array access
    zone_idx = int(zone_alloc) - 1
    handling_cost = demand * travel_distances[zone_idx]

    # 4. Capacity Penalty Check
    peak_volume = replenish_qty * item_volume
    penalty = 0.0
    if peak_volume > zone_capacities[zone_idx]:
        # High penalty multiplier to reject configurations that overflow the zone
        penalty = (peak_volume - zone_capacities[zone_idx]) * 10000.0

    return holding_cost + ordering_cost + handling_cost + penalty

def run_stockhouse_pso(demand, holding_rate, setup_cost, travel_distances, zone_capacities, item_volume):
    # PSO Hyperparameters
    swarm_size = 30
    max_iter = 50
    w = 0.7   # Inertia weight
    c1 = 1.5  # Cognitive coefficient
    c2 = 1.5  # Social coefficient

    total_zones = len(zone_capacities)

    # Bounds configuration
    min_q, max_q = 1.0, float(demand * 2)     # Order quantity bounds
    min_r, max_r = 0.0, float(demand * 0.5)   # Reorder point bounds

    # Initialize particle positions and velocities
    # Using arrays to track properties for the 30 distinct particles
    p_replenish = np.random.uniform(min_q, max_q, swarm_size)
    p_reorder = np.random.uniform(min_r, max_r, swarm_size)
    p_zone = np.random.randint(1, total_zones + 1, swarm_size)

    v_replenish = np.zeros(swarm_size)
    v_reorder = np.zeros(swarm_size)

    # Initialize Personal Bests
    pb_replenish = np.copy(p_replenish)
    pb_reorder = np.copy(p_reorder)
    pb_zone = np.copy(p_zone)
    pb_costs = np.zeros(swarm_size)

    for i in range(swarm_size):
        pb_costs[i] = evaluate_cost(
            p_replenish[i], p_reorder[i], p_zone[i],
            demand, holding_rate, setup_cost, travel_distances, zone_capacities, item_volume
        )

    # Initialize Global Best
    g_best_idx = np.argmin(pb_costs)
    g_replenish = pb_replenish[g_best_idx]
    g_reorder = pb_reorder[g_best_idx]
    g_zone = pb_zone[g_best_idx]
    g_best_cost = pb_costs[g_best_idx]

    # Main Metaheuristic Optimization Loop
    for _ in range(max_iter):
```

```
        for i in range(swarm_size):
            r1, r2 = random.random(), random.random()

            # 1. Update continuous velocity vectors
            v_replenish[i] = (w * v_replenish[i]) + (c1 * r1 * (pb_replenish[i] - p_replenish[i])) + (c2 * r2 * (g_replenish - p
            v_reorder[i] = (w * v_reorder[i]) + (c1 * r1 * (pb_reorder[i] - p_reorder[i])) + (c2 * r2 * (g_reorder - p_reorder[i

            # 2. Move positions
            p_replenish[i] += v_replenish[i]
            p_reorder[i] += v_reorder[i]

            # 3. Discrete zone reallocation logic using probabilistic swaps
            if random.random() > 0.5:
                p_zone[i] = pb_zone[i]
            elif random.random() > 0.5:
                p_zone[i] = g_zone

            # 4. Clamp continuous variables to valid, realistic limits
            p_replenish[i] = max(min_q, min(p_replenish[i], max_q))
            p_reorder[i] = max(min_r, min(p_reorder[i], max_r))

            # 5. Evaluate the fitness of the new system position
            current_cost = evaluate_cost(
                p_replenish[i], p_reorder[i], p_zone[i],
                demand, holding_rate, setup_cost, travel_distances, zone_capacities, item_volume
            )

            # 6. Check for personal improvement
            if current_cost < pb_costs[i]:
                pb_replenish[i] = p_replenish[i]
                pb_reorder[i] = p_reorder[i]
                pb_zone[i] = p_zone[i]
                pb_costs[i] = current_cost

        # 7. Check for global swarm level improvement
        current_best_idx = np.argmin(pb_costs)
        if pb_costs[current_best_idx] < g_best_cost:
            g_replenish = pb_replenish[current_best_idx]
            g_reorder = pb_reorder[current_best_idx]
            g_zone = pb_zone[current_best_idx]
            g_best_cost = pb_costs[current_best_idx]

    return g_replenish, g_reorder, g_zone, g_best_cost

if __name__ == "__main__":
    print("=== Stock House Optimization Tool (PSO) ===\n")

    # Gather product specifications
    demand = float(input("Enter Annual Unit Demand (e.g., 5000): "))
    holding_rate = float(input("Enter Unit Holding Cost per Year (e.g., 2.50): "))
    setup_cost = float(input("Enter Order Setup Cost / Delivery fee (e.g., 150): "))
    item_volume = float(input("Enter Cubic Volume footprint per unit of stock (e.g., 0.5): "))

    # Configure warehouse layout zones dynamically
    num_zones = int(input("\nHow many storage zones are in the warehouse? "))

    travel_distances = []
    zone_capacities = []

    print("\nConfigure each zone's spatial parameters:")
    for i in range(num_zones):
        dist = float(input(f"  Zone {i+1} - Distance to shipping dock in meters: "))
        cap = float(input(f"  Zone {i+1} - Total cubic volume storage capacity limit: "))
        travel_distances.append(dist)
        zone_capacities.append(cap)

    print("\n[Running Particle Swarm Optimization Swarm Engine...]")

    # Execute convergence
    best_q, best_r, best_z, min_total_cost = run_stockhouse_pso(
        demand, holding_rate, setup_cost, np.array(travel_distances), np.array(zone_capacities), item_volume
    )

    # Output optimized strategies
    print("\n" + "="*45)
    print("         OPTIMIZED SOLUTION FOUND")
    print("="*45)
    print(f"Optimal Order Quantity (Q) : {best_q:.2f} units")
```

```
    print(f"Optimal Reorder Point (R)  : {best_r:.2f} units")
    print(f"Allocated Warehouse Zone   : Zone {int(best_z)}")
    print(f"Minimized Total System Cost: ₹{min_total_cost:.2f}/year")
    print("="*45)
```

```
=== Stock House Optimization Tool (PSO) ===

Enter Annual Unit Demand (e.g., 5000): 10000
Enter Unit Holding Cost per Year (e.g., 2.50): 5
Enter Order Setup Cost / Delivery fee (e.g., 150): 200
Enter Cubic Volume footprint per unit of stock (e.g., 0.5): 1

How many storage zones are in the warehouse? 2

Configure each zone's spatial parameters:
  Zone 1 - Distance to shipping dock in meters: 4000
  Zone 1 - Total cubic volume storage capacity limit: 5000
  Zone 2 - Distance to shipping dock in meters: 3000
  Zone 2 - Total cubic volume storage capacity limit: 6000

[Running Particle Swarm Optimization Swarm Engine...]

=============================================
         OPTIMIZED SOLUTION FOUND
=============================================
Optimal Order Quantity (Q) : 894.44 units
Optimal Reorder Point (R)  : 0.00 units
Allocated Warehouse Zone   : Zone 2
Minimized Total System Cost: ₹30004472.14/year
=============================================
```